Acoustic Tracer

Authors

GitHub Repository Link: Acoustic Tracer

Team Members:

- Alex Wright - Simulation Core / Front-end
- Patryk Mrozek - Simulation Core
- Eoghan Murphy - Simulation Core / Optimization
- Michael McCarthy - Front-end

Introduction

Our project 'Acoustic Tracer' is a three-dimensional, acoustic visualiser that allows users to visualise sound travelling throughout a modelled environment as a heat map. At the core it is a C library, where the user can read in a `.glb` file (3D Model File), insert (a) speaker(s), specify simulation settings, and receive back a heat map of how the sound travelled through the environment over time. Our final project extends it into a web-based application that renders this heat map and makes the configuration of the scene and simulation more user-friendly, while also allowing more technical users to achieve their desired configuration. A user can create an account, upload `.glb` model files, view the models in a 3D scene-viewer, configure the scene and simulation settings, and finally run the simulation, returning a heat map which they can play through and replay at a later time if desired. The aim of this project was to create a unique software that could be used by architects, acoustic specialists, or anyone who desires to model how sound travels through their environment. This was also a passion project to explore the potential and concept of ray-tracing, along with creating a software that people would actually use.

Previous Works

Prior to beginning development, we surveyed the existing landscape of acoustic simulation tools to understand what was already available and where gaps lay.

ODEON[1] is one of the most established tools in room acoustic simulation. It uses a combination of the image-source method and a modified ray-tracing algorithm to predict, illustrate and auralise the acoustics of 3D environments. It is a mature, commercially licensed desktop application used widely by acoustic engineers and architects. However, it is closed-source, expensive, and desktop-only, requiring a hardware dongle for licensing. This made it unsuitable as a reference point for an open, browser-accessible tool.

CATT-Acoustic[2] is another commercial desktop application for room acoustics simulation, also using ray-tracing based methods. Like ODEON, it is a proprietary, paid, desktop-only product targeted at professional acoustic consultants.

I-Simpa[3] is an open-source GUI developed by Université Gustave Eiffel for hosting 3D numerical acoustic propagation codes. It supports ray-tracing and sound-particle tracing methods and is the closest open-source equivalent to ODEON. However, it is still a desktop application, requires separate numerical code plugins to function as a simulator, and is not designed to be embedded in or accessed via a web interface.

None of the tools we found offered a browser-based interface, real-time 3D visualisation of sound energy propagation as a voxel heat map, or a clean separation between a portable C simulation core and a platform-agnostic frontend. The volumetric, temporal voxel grid approach we implemented, where each

[1] ODEON Room Acoustics Software
[2] CATT-Acoustic
[3] I-Simpa

voxel independently records a time-binned energy history, does not appear to be a design used by any of the tools above, which typically output room impulse response parameters or auralisation rather than a spatial energy distribution over time.

Architecture

Upon starting the project our first task was to set up the shared GitHub repository where our code was to be hosted. This was an involved process with the whole team, as we wanted to ensure a proper workflow with professional version control standards. We set up an organisation, as to not have a single person hosting the repository. We then created the repository `AcousticTracer` where we each forked it, giving ourselves a personal 'copy' of the repository to serve as our `origin`. We then added a remote `upstream` where we could merge changes from our personal forks to the original repository located in the organisation. This allowed us to each work independently on features in parallel and then push these changes into a shared repository. We configured approval rules to ensure that members could not approve their own pull requests, and that each pull request required at least one approval before merging into the main branch. Finally, we implemented 'issues' that each member could view and mark as completed once implemented. This allowed us to keep a 'to-do' list, that we could delegate tasks for, and link specific commits to once finished.

Once the GitHub had been configured the next step was to work on the C library specification, since that is the core essence of the project. The C Library will henceforth be referred to as the 'core' of the project. The members working on the core of the project began at the 'top' of the core, creating the specification and outline of the public API presented by the core to the end user. Our initial scope of the project included a 'command buffer' that the output would be written to. The idea behind the command buffer was that the output of the simulation would be instructions for any graphics engine (OpenGL, Vulkan, etc.), on how to draw our heat map. This was in the hopes to create a truly independent, graphics library agnostic implementation, but this was later scrapped due to the complexity. We decided on creating a consistent communication standard instead, that could be parsed by any frontend, or visualisation tool. This communication standard will be discussed at a later stage.

The architecture / workflow we envisioned for the user was as follows:

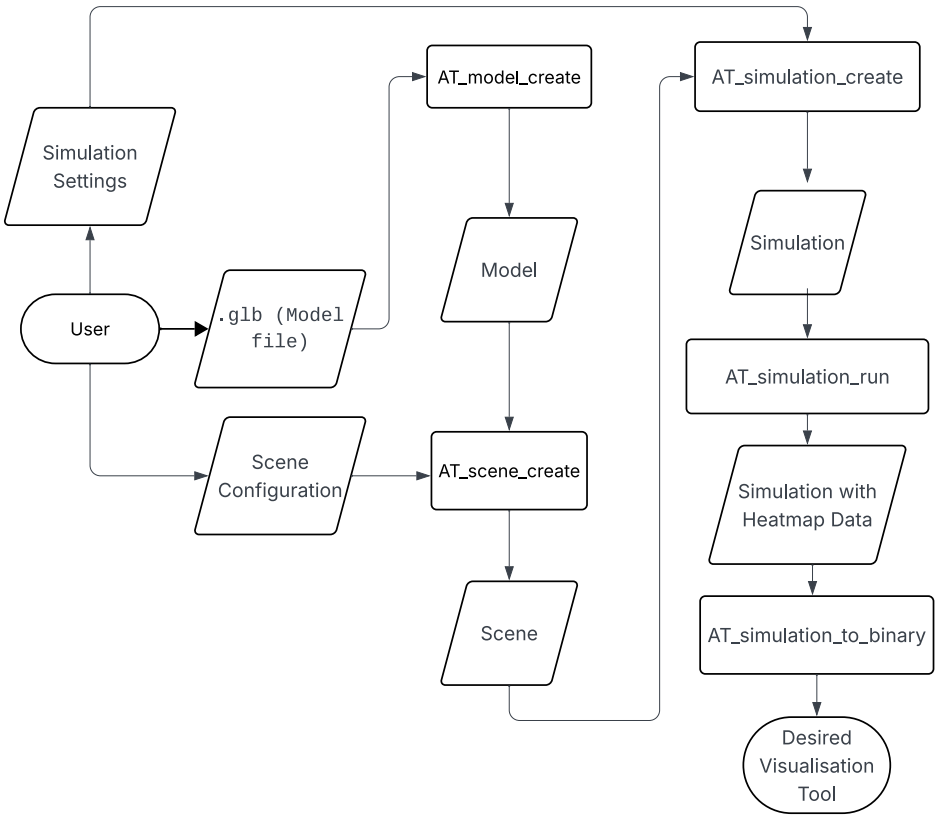

Figure 1: Flowchart of final architecture

The core of the project was written in C, over other languages, for a few key reasons. Firstly, C is extremely performant, which is crucial for our application since it is extremely computationally heavy. A key feature for us was the library `raylib` which C provides as an external library, and this allowed us to visualise our core during development. Finally we have been using dynamically typed languages such as Python and JavaScript during our degree, and we wanted to each improve our programming skills with a statically typed language. Since C is devoid of object-oriented features like classes and methods, the workflow for our C library must follow a certain structure. The user of the C library must create `struct` instances and pass these to functions that alter them. However users of the included frontend need not worry about the implementation. Each of the functions, `structs`, and data types that are publicly available to the end user are forward declared in the file `at.h` which the user of our library can include in their project. This file includes all the function signatures, along with `struct` member descriptions to make it easy for the user to understand the data flow, as well as the use of our library. These forward declarations are fully defined in `at_internal`, or in their own respective `at_*.c` files, as to abstract the implementation away from the user.

The specification of the project involved the design of the following structures and data-types (each prefixed with `AT_` as part of our core library (Acoustic Tracer)):

The frontend, as mentioned is our method of creating a universal visualiser for the heat map of the model, while remaining platform agnostic (i.e. usable on MacOS, Linux, Windows, iOS, Android, etc.). Initially, during the project specification phase, we designed a communication standard which we stuck with throughout the duration of the project, and it worked well. However, while the structure of the data remained the same, the data type had to change, which will be discussed later. We decided to create a server in C, that exposed an endpoint `/run` to the user, where they can send the scene and simulation configuration, and receive the heat map data as the server response. The frontend, could then make a

HTTP request to the server which is easily performed on any language.

Simulation

Model

Creating the model was inherently the first step of our project, as that is the first step to the user flow. During our initial implementation specification design, we settled on the following model definition:

```
struct AT_Model {
    AT_Vec3 *vertices;
    AT_Vec3 *normals;
    uint32_t *indices;
    uint32_t *triangle_materials;
    size_t vertex_count;
    size_t index_count;
};
```

This creation of the `AT_Model` struct was possible with the library `cgltf`[4], which is a single-file/stb-style C glTF loader and writer. This library gives us the ability to parse the `.glb` file the user provides, resulting in access to the vertices, nodes, indices, and transformation matrices for each of the vertices rotation, scale, and quaternion. Each of these matrices encode data describing each node in relation to the world. The rotational matrix stores information about how to rotate a node, the scalar matrix shows how to scale a node, and finally the quaternion matrix encodes information about an axis-angle rotation around an arbitrary axis. Initially the `AT_model_create()` function only extracted the raw vertex data, which worked in the beginning. But as the need for more complex models arose, we had to alter our approach to make the use of `cgltf_node` attributes with the vertex data, combined with the parent and world transformation matrices, to give the vertex position and state in relation to the world.

Instead of naively extracting the raw vertex, index and normal data from the `.glb`, we must apply the parent and world rotation, scale, and quaternion matrices to each node. This is achieved by 'walking' up the node hierarchy, and multiplying each of the three matrices, until we arrive at the desired node. In this way, the three matrices accumulate across the parents, giving us an exact state of the node in relation to the 'world'.

A model is a struct, composed of each of the following members:

- An array of vertices, each with an `x`, a `y`, and a `z` component, which represents a point in 3D space.
- An array of normals, which is the direction that the triangle 'faces'. Represented as a three-dimensional vector.
- An array of indices, which outlines the order into which to draw lines from each vertex, creating the triangles of the model.
- An array of materials for each of the triangles, where the array is `index_count / 3` in length and the entry at `triangle_materials[t]` is the material for the triangle at `t`.
- A number representing the total amount of vertices in the model
- A number representing the total amount of indices in the model

Scene & Simulation

Creating the scene with `AT_scene_create()` is a simple aggregation of the `AT_SceneConfig` configuration, model (environment) pointer, and creation of the AABB (axis-aligned bounding box) of the model. Similarly the creation of the simulation with `AT_simulation_create()`, is another aggregation of the

[4] cgltf Library

simulation settings (`AT_Settings`), calculation of the voxel grid dimensions, the allocation of the voxel grid, and the transfer of ownership of the `AT_Scene` pointer (similar to the scene transferring ownership of the model pointer).

Core Simulation Phases

Our core C library, as mentioned, involves the simulation of sound as rays throughout a three-dimensional environment. This implementation involves two main steps:

1. Ray Bounce Tree
2. DDA (Digital Differential Analyser) Voxel Sweep

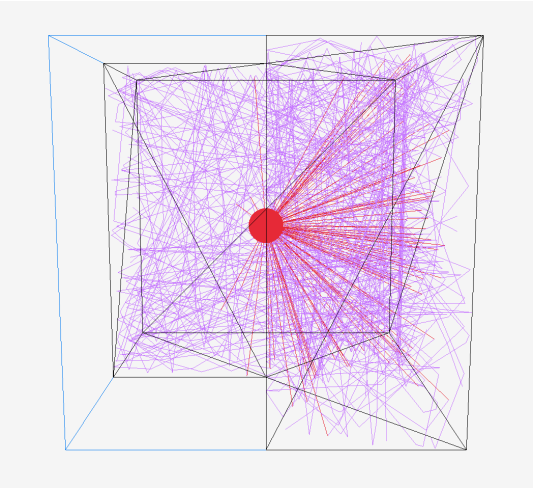

Figure 2: Phase 1 — Ray Bounce Tree

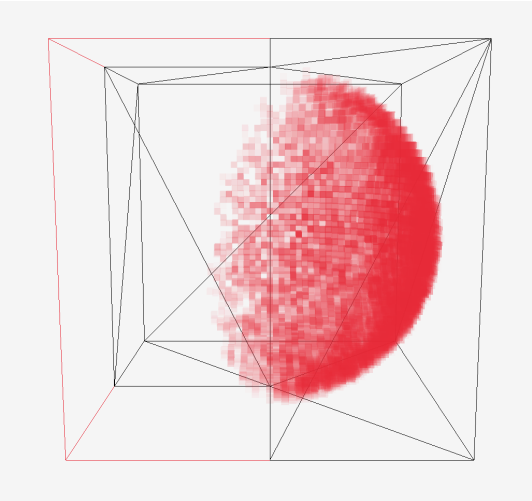

Figure 3: Phase 2 — DDA Voxel Sweep

Rays

A ray, for our purposes in this project, is a 3D representation of a line that extends infinitely from a source point (represented as a three-dimensional vector `{x, y, z}`), in a given direction (also represented as a three-dimensional vector `{dx, dy, dz}`, where each of the components is the change of the position along the x, y, and z axes, respectively).

Our first step in the implementation of the project was to emit rays from the 'speaker' (which will be hereby referred to as the 'source'). Let's assume for the moment that we have the scene configuration and the simulation configuration which are defined as follows:

```
typedef struct {
  const AT_Source *sources;
  uint32_t num_sources;
  AT_MaterialType material;
  const AT_Model *environment;
} AT_SceneConfig;

typedef struct {
    AT_Vec3 position;
    AT_Vec3 direction;
} AT_Source;
```

Where the `environment` is the `struct` of the 3D model the user specifies, and the source is composed of a position and direction (similar to the vector). We initialise rays using `AT_ray_init()` at the position of the source, setting their initial direction and position to those of the source they come from. Since we then want to create a realistic simulation of how these rays interact (bounce, scatter, and absorb) throughout the environment, we must implement each phenomena associated with these 'sound rays'.

The first phenomena of the rays we implemented was reflection. This is where a ray 'bounces' off a surface, creating a new ray with an origin of the point of intersection of the surface, and a 'reflected' direction. Before we implemented reflection, we first had to implement the intersection between a ray and a triangle. Since our model is composed of entirely triangles, it is important to define what it means for a ray to intersect with a triangle.

This is where we used the Möller-Trumbore ray/triangle intersection algorithm[5]. This allows us to compute whether a ray intersects with a triangle without having to pre-compute the plane equation of the plane containing the triangle.

Once we were able to get whether a ray intersected with a triangle, the next issue was to handle what actually happens when the ray intersects with the triangle. Initially we implemented a `hit_list` which was given to each ray, that tracked each point on any triangle that the ray intersected with. We must keep track of every single point of intersection, and then only 'handle' the point of intersection closest to the ray origin, as this would be the 'first' intersection, regardless of the order of the triangle checks. This caused problems with memory management, which is important in C. Since we didn't know until runtime, how many times a ray would intersect with any of the triangles, we didn't know how big to make the `hit_list` array. This introduced the concept of the dynamic array, which functions like lists do in Python and JavaScript, where the size of the list increases, if appending an element exceeds the capacity of the list. Once we had computed the closest point of intersection of each ray, we could then initialise a new ray with the origin of the point of intersection and the 'reflected' direction.

To combat the memory management associated with the `hit_list` approach of calculating the closest point of intersection, we pivoted to using a linked list approach for the ray implementation. Each ray would have a `child` ray that is the resultant ray after intersection, scattering, reflection etc. Upon the first intersection of a ray we would initialise a new ray with the computed origin and direction, and only update its direction and origin upon subsequent intersections, only if the distance from that point of intersection was less than the distance from the current rays origin to its child. This greatly simplified the computation, while also introducing an hierarchy among the rays, with a parent/child relationship. We could also easily navigate this ray hierarchy using linked list traversal methods. Figure 2 shows a visualisation of the ray bounce tree in action, where each red line represents the `parent` rays and the purple lines represent all of the `child` rays.

Another ray phenomena implemented in our project is material absorption. Since this is not a concern of the end user of the library, this is defined in our `at_internal.h` header file where we declare functions, enumerations, and structs, to be used internally. For the absorption (and later scattering) we implemented, we decided to declare a table of materials along with their coefficients:

```
static const AT_Material AT_MATERIAL_TABLE[AT_MATERIAL_COUNT] = {
    [AT_MATERIAL_CONCRETE] = {.absorption = 0.02f, .scattering = 0.10f},
    [AT_MATERIAL_PLASTIC] = {.absorption = 0.03f, .scattering = 0.05f},
    [AT_MATERIAL_WOOD] = {.absorption = 0.10f, .scattering = 0.20f},
};
```

The scattering coefficient determines the probability that a ray, upon hitting a surface, is redirected diffusely in a random direction within the hemisphere above the surface normal, rather than following the

[5]Möller-Trumbore Intersection Algorithm

angle of perfect specular reflection. A rough concrete wall scatters more than a smooth plastic surface, which is reflected in the coefficients above.

Upon intersecting with a triangle, (whose material has been decided during `AT_scene_create()` as stated above), we can calculate the rays resultant energy modelled with the formula:

```
child->energy = ray->energy * (1.0f - triangle_material.absorption_coefficient);
```

This accurately uses the absorption coefficient for each material to alter the resultant energy of the `child` ray, created from the intersection.

The initial direction of each ray is also not arbitrary. Rather than emitting rays uniformly in all directions, we used cosine-weighted hemisphere sampling, implemented in `AT_sample_cosine_hemisphere()` in `at_utils.h`, with reference to the sampling method described in the PBR Book [6]. The function constructs an orthonormal basis from the source's direction vector, and samples a direction in the hemisphere above that surface using two uniform random floats. The polar angle is derived as `acos(sqrt(1 - u))` rather than `acos(u)`, and it is this square root that produces the cosine weighting as a consequence of the geometry of the sphere, biasing rays towards the forward direction of the source and away from grazing angles. The same function is also used when a ray scatters diffusely off a surface.

Voxels

The second phase of the simulation is the DDA voxel sweep. A voxel is a volumetric pixel, and in our case, a voxel is a dynamic array defined as follows:

```
typedef struct {
    float *items;
    size_t count;
    size_t capacity;
} AT_Voxel;
```

This structure allows us to use our general purpose dynamic array functionality defined in `at_utils.h`. `items` is a pointer to an array of floats, which we call "bins". The concept of a bin is essential to the temporal aspect of the simulation. Each bin stores the total amount of sound energy that arrived in the voxel during a specific time interval. Think of each voxel as having a timeline, where each slot on the timeline records how much sound energy passes through the voxel within that window. During the replay, the bins are revealed sequentially.

In this phase, a voxel grid stores the spatial distribution of sound energy across the scene. The size of the voxels is user-specified, this size determines the resolution of the simulation. Smaller voxels produce a more detailed heat map but require significantly more memory and computation. The world dimensions of the voxel grid are calculated by subtracting the max and min value of the scenes AABB, which is the smallest possible box that encapsulates the entire model. These world dimensions are then used to calculate the grid dimensions by dividing the world dimensions by the user-specified voxel size, which gives `grid_x`, `grid_y` and `grid_z`.

Rather than allocating a 3-dimensional matrix, the voxel grid is stored in a 1-dimensional array of `AT_Voxel` types. A voxel at position (x, y, z) in the grid can be accessed using the formula `z * grid_y * grid_x + y * grid_x + x`. A contiguous block of memory is more efficient to allocate and iterate over than nested pointers.

During the planning phase of the project, our initial design for the voxel grid used a fixed number of bins per voxel, derived from a predetermined replay length:

[6] PBR Book

```
typedef struct {
    float energy_bins[NUM_BINS];
} Voxel_t
```

This approach required knowing the total simulation duration upfront in order to allocate the correct number of bins, though in practice this was not always possible. The total duration of a simulation depends on how far the rays travel before their energy drops below the termination threshold, which in turn depends on the geometry of the scene and the configuration of the source. The amount of times a ray bounces and how it loses energy throughout the environment is not something we can know until the simulation is run. We therefore moved to the dynamic array approach described above, where each voxel's bin array grows dynamically as energy is deposited into new time frames.

Every ray generated in the first phase of the simulation is then traversed using the Digital Differential Analyzer (DDA) algorithm, implemented with reference to Amanatides and Woo's *A Fast Voxel Traversal Algorithm for Ray Tracing* algorithm[7]. A naive approach to this problem might sample points along a ray at fixed intervals, but this risks skipping voxels entirely if they are only grazed by a ray, and also opens up the possibility for a voxel to be visited multiple times. The DDA algorithm allows us to track how far along a ray segment we need to travel to cross the next voxel boundary per axis, which is tracked by the variable `t_max`. At each step, the axis with the smallest `t_max` is advanced. This guarantees that every voxel the ray segment passes through is visited exactly once, regardless of the ray's direction or the size of the voxels.

For each voxel the ray crosses, an amount of energy is deposited into that voxel. This energy deposit is weighted by three physical factors. First, the length of the ray segment inside the voxel, a ray travelling a longer path through a voxel contributes more energy to it. Second, inverse square attenuation with total distance `d` from the source, modelling how sound naturally loses intensity over distance. Third, air absorption, modelled as `exp(-k * d)`, where `k` is the air coefficient, which accounts for energy lost to the medium itself as the 'wave' propagates. The result of these three factors combined is a single float value that is added to the voxel's current bin.

The current time of the simulation `t` is calculated as `d / v` where `v` is the current simulation speed. Throughout the development of this project, `v` was one of two values. First being 343 m/s, which is the speed of sound, and an arbitrary slower speed that was used while building the visualisation, to make the movement of the sound energy through the heat map easier to observe. The current "bin" index is then calculated with `floor(t / bin_width)`, where `bin_width = 1 / fps`. This is the design decision that makes the output a temporal result rather than a static energy snapshot. Each voxel records how much energy arrived, as well as when it arrived. Each voxel's bin array grows dynamically since at allocation time the duration of the simulation is not yet known. Figure 3 shows the resulting voxel heat map for the same scene, where the colour intensity of each voxel represents the amount of sound energy deposited into it across all time bins.

Optimisation

In order to get the central workings of our simulation underway, we went with the naive approach of determining if a ray intersected with any triangles, by checking each ray against all triangles in the scene. This approach is incredibly computationally intensive, as any real-world model would contain hundreds of thousands, if not millions of triangles, and our simulation would require tens of hundreds of thousands of rays, which very quickly becomes infeasible for local computation purposes.

As such, once the ray phase of the simulation had been implemented, the team began working on a better intersection solution. The industry solution to this problem is to break the scene up into a tree that can

[7] Amantides-Woo Voxel Traversal Algorithm

be traversed, exponentially reducing the number of required triangle intersection tests.

Before we can get around to discussing scene tree implementations, we must first, talk about how to split a scene, to produce a useful tree. In order to create a useful tree from a given scene, we must determine a series of "optimal" splitting points, in order to minimise the size and overlap of resultant bounding boxes.

Unfortunately, as in many areas of computer science, these two criteria cannot both be perfectly optimised at the same time. This is where object vs spatial based splits come into play. A scene can be split in two ways, using an object-based, or a spatial-based split[8].
An object split means splitting a scene based on object boundaries, whereas a spatial split, involves splitting based on space alone, potentially resulting in a given object being in multiple splits. The advantage of an object-based split is the removal of the possibility of objects being "double counted" in splits, with the cost of overlapping bounding boxes; introducing the need to check multiple bounding boxes to find the nearest triangle. On the other hand, spatial splits can result in a more optimal bounding box, with the extra cost of double counting objects.

We chose to split our scene using object-based methods, as these often involve simpler algorithms, and are best suited for scenes with similarly sized, well-distributed triangles, which fit for the simulation's intended scenes.
This meant using a Bounding Volume Hierarchy (BVH) to represent our scene tree.
A BVH is a representation of a scene, built through a series of object-based splits, resulting in a binary tree, where leaf nodes are objects, and internal nodes are bounding boxes for the aggregation of all it's descending nodes. Ray-triangle are then carried out by traversing the tree, checking if the ray intersects with a node's bounding box, which determines if it's children are checked or discarded.
There are a number of decisions involved when building a BVH, the first being, how to represent the bounding box of a triangle. The most common representations are a sphere or a cuboid (also known as an Axis-Aligned Bounding Box). We went with an AABB representation as while the intersection test is more computationally intensive, the resultant bounding box is much tighter for most triangles when using an AABB, than with a sphere.

Our next decision was what implementation of a BVH would we follow. After researching potential implementations, such as, Meister's PLOC[9], or Wald's "binned" approach[10]. We decided upon a lesser known implementation dubbed "Bonsai"[11].
This implementation was chosen for a number of reasons:

1. The implementation was created with the CPU in mind,
2. The implementation balanced efficient build times, with optimal tree quality, and
3. The paper kept the implementation vague which allowed us to develop our own algorithm.

Our implementation of the Bonsai algorithm is not a one-to-one replica, as no official pseudo-code could be found. As such we relied on the paper's technical description along with previous BVH knowledge from reading other implementation pseudo-code.

The *original* Bonsai algorithm is composed of 5 steps:

1. Compute the midpoint of each triangle,
2. Split the triangles into smaller groups based on their midpoints,
3. Generate mini BVH trees based on triangle groups,
4. Prune mini trees to find better optimised subtrees,
5. Merge all mini trees into a singular BVH entity.

[8]Space Division Algorithms Paper
[9]PLOC Paper
[10]Binned BVH Paper
[11]Bonsai Paper

Our implementation closely follows the first three steps, but diverges from there; as such, we will briefly discuss each step, but only go further into detail on the relevant steps.

C Library

As mentioned before, C has no classes or namespaces. Building a library with a clean public interface that hides internals therefore requires deliberate design choices. The following describes the pattern we used to achieve this.

Public API (`at.h`)

The only file a user of our library needs to include is `at.h`. This file contains all of the function signatures, type definitions and enum declarations that are publicly available. The three main types of the library are declared here as incomplete types:

```
typedef struct AT_Model AT_Model;
typedef struct AT_Scene AT_Scene;
typedef struct AT_Simulation AT_Simulation;
```

Because they are incomplete types, a user can hold a pointer to them but cannot dereference them to access their members directly. The full struct definitions are never visible outside the library. This also allows us to change the internal layout of any struct without breaking any code that uses the library, because the user only every holds a pointer to an incomplete type, the compiler never needs to know the size of the struct itself.

All publicly available functions follow the same naming convention, prefixed with `AT_` and the name of the type they operate on:

```
AT_Result AT_model_create(AT_Model **out_model, const char *filepath);
void      AT_model_destroy(AT_Model *model);

AT_Result AT_scene_create(AT_Scene **out_scene, const AT_SceneConfig *config);
void      AT_scene_destroy(AT_Scene *scene);

AT_Result AT_simulation_create(AT_Simulation **out_simulation,
                               const AT_Scene *scene,
                               const AT_Settings *settings);
AT_Result AT_simulation_run(AT_Simulation *simulation);
void      AT_simulation_destroy(AT_Simulation *simulation);
```

The `AT_` prefix was a deliberate decision made during the planning phase of this project. Without it, names like `model_create` or `simulation_run` could easily clash with function names from other libraries, producing confusing linker errors. The prefix sort of acts like a manual namespace.

A typical usage of the library follows a consistent create, use, destroy cycle:

```
AT_Model *model = NULL;
if (AT_model_create(&model, "room.glb") != AT_OK) {
    fprintf(stderr, "Failed to load model\n");
    return 1;
}

AT_Source source = {
    .direction = {{0.0f, 1.0f, 0.0f}},
```

```
    .position = {{0.0f, 0.0f, 0.0f}}
};

AT_SceneConfig config = {
    .environment = model,
    .sources = &source,
    .num_sources = 1,
    .material = AT_MATERIAL_CONCRETE,
};

AT_Scene *scene = NULL;
if (AT_scene_create(&scene, &config) != AT_OK) {
    fprintf(stderr, "Failed to create scene\n");
    return 1;
}

AT_Settings settings = {
    .fps = 60,
    .num_rays = 1000,
    .voxel_size = 1.0f
};

AT_Simulation *sim = NULL;
if (AT_simulation_create(&sim, scene, &settings) != AT_OK) {
    fprintf(stderr, "Failed to create simulation\n");
    return 1;
}

AT_simulation_run(sim);

AT_simulation_destroy(sim);
AT_scene_destroy(scene);
AT_model_destroy(model);
```

The user must destroy in reverse of the creation order. This is because `AT_Simulation` borrows a pointer to `AT_Scene` and `AT_Scene` borrows a pointer to `AT_Model`. The ownership convention is documented well within the internal codebase.

Internal Architecture (`at_internal.h`)

The full struct definitions for all three opaque types live in `at_internal.h`, along with any other types that need to be shared across multiple source files but should never be exposed publicly. The reason that this internal header exists, as opposed to simply defining the structs within their respective `.c` files, is that multiple source files need access to full definitions simultaneously.

The full definitions are as follows:

```
struct AT_Scene {
    AT_Source *sources;
    AT_AABB world_AABB;
    uint32_t num_sources;
    AT_MaterialType material;
```

```
    const AT_Model *environment;
    //...
};

struct AT_Simulation {
    const AT_Scene *scene;
    AT_Voxel *voxel_grid;
    AT_Ray *rays;
    AT_Vec3 origin;
    AT_Vec3 dimensions;
    AT_Vec3 grid_dimensions;
    float voxel_size;
    float bin_width;
    uint32_t num_rays;
    uint32_t num_voxels;
    uint8_t fps;
};
```

The chain of ownership can be seen directly in the struct definitions. `AT_Simulation` holds a `const AT_Scene *` and `AT_Scene` holds a `const AT_Model *`. The const keyword here signals that these are borrowed references, and that the struct is not responsible for freeing them. In contrast, `AT_Voxel *voxel_grid` inside `AT_Simulation` carries no `const`, meaning the simulation owns that data outright, and `AT_simulation_destroy` is responsible for freeing it. This is also why destroy calls must be made in reverse order of creation. `AT_Simulation` must be freed before `AT_Scene`, because freeing the scene while the simulation still holds a pointer to it would leave the simulation with a dangling reference.

Error Handling (`AT_Result`)

Every function in the library that could potentially fail returns an `AT_Result`:

```
typedef enum {
    AT_OK = 0,
    AT_ERR_INVALID_ARGUMENT,
    AT_ERR_ALLOC_ERROR,
    AT_ERR_NETWORK_FAILURE
} AT_Result;
```

To combat the fact that C can't throw exceptions, the convention is to return an error code that the caller must explicitly check. This is particularly important for our library since the simulation involves a large number of heap allocations, any of which could fail. Detecting these failures early produces a clear error message, rather than a segmentation fault.

`at_result.h` also provides a small helper function `AT_handle_result()` which prints the error type and a custom message to `stderr`, used throughout development to quickly surface allocation ad argument errors without having to write a switch case every time.

Communication between the Core and the Front-End

With our exemplar frontend web application, the workflow consists of the frontend sending the following data to the core backend:

```
{
  "filepath": "<path_to_glb>",
```

```
  "voxel_size": "<size_of_voxel {float}>",
  "material": "<model_material>",
  "fps": "<fps {uint8}>",
  "num_rays": "<num_rays {uint32}",
  "source": {
    "position": ["<pos_x {uint32}>", "<pos_y {uint32}>", "<pos_z {uint32}>"],
    "direction": ["<dir_x {uint32}>", "<dir_y {uint32}>", "<dir_z {uint32}>"]
  }
}
```

Since this JSON is quite minimal, it is acceptable to send as-is.

The backend for our application is centred around a C server whose implementation is defined in `backend/at_net.c`. This involves setting up a TCP socket (which essentially behaves like an API endpoint in the eye of the user). It listens for incoming connections, accepts only `POST /run` requests, parses the incoming configuration and settings, runs the ray-tracer, converts the frame data (voxel bins) to binary, and finally writes the binary stream as the response to the client. The frontend receives this stream, parses the result, and constructs the visualisation of the heat-map on the client-side.

During our initial design phase, we outlined the communication standard for the voxel data (bins) as shown:

```
{
  "1": [{"1": 24}, {"3": 34}, {"6": 32}, {"13": 45}, ...]
  "2": [...]
  .
  .
  .
  "n": [...]
}
```

Here, the first key is the frame number, and the value is a list of `key: value` pairs where the key is the voxel index, and the value is the energy of the voxel at that current frame. We found that this was an easy to understand communication standard, while reducing the amount of total tokens required to transmit. Furthermore it is natively JSON, so it is easy to parse on the frontend application. In the final iteration we prefixed the frame number with `frame_` to distinguish between the frame number and the voxel index.

On the backend, we used the `cJSON` library [12], to parse the configuration received from the frontend.

With this communication standard, we avoid sending unnecessary data to the client, only sending voxels with their energy over a minimum threshold defined internally.

Frontend

Introduction to Frontend

The AcousticTracer project, pairs a C simulation engine (the core) with a browser-based frontend, which configures, stores, and visualises the simulation. Although the team member's role on this project was frontend engineer, the nature of the work bore little resemblance to conventional frontend development. The typical concerns of a UI/UX-focused role were secondary throughout. The primary challenges were technical: writing per-frame transform matrices directly into GPU-backed buffers, clamping 3D coordinates to axis-aligned bounding boxes during drag interactions, and orchestrating an asynchronous pipeline spanning two independent backends.

[12] cJSON library

The focus on these technical aspects was not a deliberate de-prioritization but was an accurate reflection of where the complexity lay. The frontend's contributions to the project was not measured in how visually appealing the frontend looked but in its ability to bridge the gap between a C simulation engine and a browser-based 3D visualisation.

Goals

The frontend has three responsibilities that we discussed and outlined early in development but did not yet know how to tackle, and each came with distinct technical demands:

1. **Configure and submit** — The user uploads a `.glb` room model, sets simulation parameters (voxel size, ray count, FPS, surface material), and interactively places a sound source by adjusting its position inside the 3D scene. The configuration is sent to the C backend as JSON matching the communication standard defined earlier.
2. **Decode and store** — Initially the C backend returns results in JSON format but due to performance limitations we had incorporate a custom `ATRB` (Acoustic Tracer Result Binary) binary format. The frontend decodes this into typed arrays, uploads it to Appwrite file storage, and caches the parsed result so revisiting a completed simulation is instant.
3. **Render and replay** — The decoded frames are visualised as a 3D voxel heat map overlaid on the room model. Consider a modest room of 8m × 5m × 5m with a voxel size of 0.1m, that produces 80 x 50 x 50 = 200,000 voxels, and a typical simulation might fire 100,000 rays. Each of those 200,000 voxels requires per-frame position and colour updates at the configured frame rate. These are not even worst-case scenario numbers, yet the frontend needed to be able to provide a smooth and interactive experience.

Technology Decisions

Before delving into the architectural design and implementation, this subsection briefly describes the technologies used and why they were chosen.

React React was chosen as the UI framework because the frontend developer had invested time developing skills with it before and during the early stages of the project, completing Scrimba's introductory course and the majority of the advanced React courses, as well as their TypeScript course. That preparation provided enough fluency with React's component model, hooks, and ecosystem to be productive from the first week.

TypeScript The initial skeleton was plain JavaScript. It quickly became apparent that JavaScript alone would not be sufficient, the project was hitting runtime crashes caused by misspelled prop names and `undefined` values propagating silently through the component tree, bugs that TypeScript's structural type system catches at compile time. The frontend developer invested time during the first three weeks of development learning TypeScript alongside developing the codebase.

Tailwind CSS

Hand-written CSS files worked fine while the project had five components. Once that count reached fifteen, issues started to arise. Tailwind eliminated this problem entirely by moving styling into utility classes located within the JSX. This co-location is where Tailwind and React complement each other naturally, because React components are self-contained, having the styling live inline as class names means a component's appearance, behavior, and structure are all visible in a single file. Tailwind v4's CSS-native `@theme` directives allowed us to define project-wide design tokens directly in `index.css`.

```
@import "tailwindcss";

@theme {
  --color-bg-primary: #2d2d39;
  --color-bg-card: #282833;
  --color-text-primary: #ffffff;
  --color-accent: #fbbf24;
  --color-button-primary: #4f46e5;
  --color-danger: #f87171;
  --color-success: #34d399;
}
```

UI Design

Towards the end of the project, after the core features of the frontend had been developed and optimised, the need for a polished UI could no longer be ignored. building these from scratch would have been a significant time investment that we did not have the luxury of. We found **shadcn/ui**, a collection of copy-and-paste React components built on **Radix UI** primitives and styled with Tailwind. This approach aligned with our existing Tailwind-based styling and allowed incremental adoption.

The Architecture

Feature Orientated Architecture

Roughly two weeks into development, the codebase was restructured from a flat component directory to a feature orientated architecture where each major feature is a self-contained directory:

```
web/src/
+-- app/              # Shell: provider composition, router, global CSS
+-- api/              # Barrel re-exports the data contracts
+-- components/       # Shared UI
+-- features/
|   +-- auth/         # Login, Register, Settings, OAuth, UserProvider
|   +-- simulation/   # Everything acoustic: API, components, hooks, routes, store
+-- lib/              # Infrastructure: Appwrite client, QueryClient, utils
+-- utils/            # Pure helpers
```

This codebase structure enforces the concept of **import direction**: feature code may import from `lib/` and `components/`, but never from another feature directly. The `auth` and `simulation` features communicate only through the provider hierarchy and through barrel exports in `api/`. This structure was well thought out and easy to navigate and understand once it was explained to the rest of the team. Similar to our C library's use of `at.h` as the single public interface, each feature exposes its functionality through a controlled set of barrel exports, keeping the internals hidden from the rest of the application.

Provider Hierarchy

In React, a **provider** is a component that makes shared data or services available to every component nested inside it, without having to pass that data down manually through props at each level of the component tree. Providers were used to compose the application's global infrastructure (error handling, loading states, data caching, authentication) into a single wrapper so that every page and component has access to these services automatically.

The application's provider stack is composed in `provider.tsx`:

```
<ErrorBoundary FallbackComponent={MainErrorFallback}>
  <Suspense fallback={<LoadingSpinner />}>
    <QueryClientProvider client={queryClient}>
      <UserProvider>
        {children}
        <ToastContainer />
      </UserProvider>
    </QueryClientProvider>
  </Suspense>
</ErrorBoundary>
```

1. **ErrorBoundary** - catches any uncaught exception, including Suspense promise rejections, and renders a recovery UI.
2. **Suspense** - displays a loading spinner while any descendant component suspends (e.g., during lazy-loaded route fetching).
3. **QueryClientProvider** - makes the TanStack Query cache available to all descendants.
4. **UserProvider** - initialises authentication state. On login and logout, it resets the Zustand SceneStore and clears the TanStack Query cache to prevent data leakage between sessions.

The Data Layer

The data layer is the set of abstractions that sit between the frontend's UI components and the two external services, the appwrite backend and the C ray-tracer, HTTP endpoint. Its purpose is to ensure that no component ever interacts with either service directly.

Type Mapping

The Appwrite database stores simulation records as flat, snake_case documents (`SimulationDocument` in `contracts.ts`). The frontend consumes them as nested, camelCase domain objects (`Simulation` in `simulation-repository.ts`). Without a clear boundary between these two representations, Appwrite's naming conventions and flat structure would leak into every component that touches simulation data.

```
function documentToSimulation(doc: SimulationDocument): Simulation {
  return {
    $id: doc.$id,
    $createdAt: doc.$createdAt,
    $updatedAt: doc.$updatedAt,
    name: doc.name,
    status: doc.status,
    userId: doc.user_id,
    inputFileId: doc.input_file_id,
    resultFileId: doc.result_file_id,
    computeTimeMs: doc.compute_time_ms,
    numVoxels: doc.num_voxels,
    fileName: doc.file_name,
    config: {
      voxelSize: doc.voxel_size,
      fps: doc.fps,
      numRays: doc.num_rays,
      material: doc.material,
      selectedSource: {
        position: { x: doc.position_x, y: doc.position_y, z: doc.position_z },
```

```
        direction: { x: doc.direction_x, y: doc.direction_y, z: doc.direction_z },
      },
    },
  };
}
```

Repository Pattern

Just as our C library hides all internal implementation behind the public API in `at.h`, we did not want any component in the frontend importing from the Appwrite SDK directly. So we introduced a `SimulationRepository` that encapsulates all Appwrite SDK interactions behind typed methods:

```
export const simulationRepo = {
  list: (userId: string): Promise<SimulationList> => { ... },
  getById: (id: string): Promise<Simulation> => { ... },
  create: (params: CreateSimulationParams): Promise<Simulation> => { ... },
  update: (id: string, params: UpdateSimulationParams): Promise<Simulation> => { ... },
  delete: (id: string, fileId: string, resultFileId?: string): Promise<void> => { ... },
  uploadFile: (file: File | Blob, name?: string): Promise<string> => { ... },
  getFileUrl: (fileId: string): string => { ... },
};
```

This means the entire Appwrite SDK could be replaced without changing a single component file, and a database schema change requires updating only the contract type and the mapper. The repository is accessed exclusively through TanStack Query hooks (`useSimulationsList`, `useSimulationDetail`, `useCreateSimulation`, etc.) that are re-exported through `api/simulations.ts`.

Query Keys and Caching

TanStack Query uses structured keys defined in `query-keys.ts` to manage caching and invalidation:

```
export const simulationKeys = {
  all:           ["simulations"] as const,
  lists:         (userID?: string) => [...simulationKeys.all, "list", userID] as const,
  details:       () => [...simulationKeys.all, "detail"] as const,
  detail:        (id: string) => [...simulationKeys.details(), id] as const,
  rayResponses:  () => [...simulationKeys.all, "rayResponse"] as const,
  rayResponse:   (fileId: string) => [...simulationKeys.rayResponses(), fileId] as const,
};
```

This structure allows for efficient cache invalidation. For example, after creating a simulation, calling `invalidateQueries()`, **refetches** the list without disturbing cached detail queries or ray responses.

The Binary Protocol

The initial design for the voxel data used JSON. While JSON was easy to understand and parse, the payload sizes for a real simulation (200,000+ voxels with 100,000+ rays) were extremely large. The ATRB binary format replaced it, and on the frontend this binary response is decoded by `parseResultBuffer()`.

Simulation Submission

When the user submits a simulation, the `useSceneActions` performs the following actions:

1. Upload the `.glb` file.

2. Create a database record.
3. Navigates to the dashboard immediately
4. Runs the ray tracer in the background.

On success, the binary result is uploaded to Appwrite storage and the parsed `RayFrame[]` is added into the TanStack Query cache so the result is available instantly when the user revisits the simulation.

The `runRaytracer` function itself is a `fetch` POST to the C backend's `/run` endpoint, sending the configuration as JSON and receiving the binary response.

The Rendering Pipeline

Three.js, React Three Fiber, and Drei

The 3D Rendering aspect of the frontend consisted of 3 layers, At the base layer sits **Three.js**. It gives us access to powerful API abstractions, which allows us to translate developer-friendly code into the complex WebGL instructions needed to render 3D graphics. The frontend aspect of this project would not have been possible without this library and we cannot stress enough the important of it. It gave us access to critical methods such as, `InstancedMesh()`, `BufferGeometry()`, `Matrix4()`, `Color()`,and `Box3()`, that were used throughout the development of this project.

On top of the base layer sits **React Three Fiber** (henceforth R3F), it is a custom React renderer that maps JSX elements to three.js objects. What makes R3F so powerful is that it lets us describe a 3D scene using the same declarative, component-based model we use for the rest of the UI. Without R3F, we would have had to manually create objects, add them to the scene, remove them on cleanup, and synchronise state between React and Three.js. R3F eliminates the need to do this entirely, adding the voxel grid to the scene is no different from adding a button to a form, it is a component that mounts, updates when its props change, and unmounts cleanly.

The third layer is **Drei**, a companion library of pre-built R3F components and hooks. Drei saved significant development time by providing these components/helpers. To name a few:

- `OrbitControls` - for camera interaction.
- `Bounds` - for automatic camera framing around the model.
- `useGLTF` - for loading `.glb` files.
- `TransformControls` - for the source placement gizmos.
- `Environment` - for scene lighting,

The abstraction provided by these three libraries was essential to completing this project. Three.js abstracted WebGL code into objects and methods we could deal with, R3F abstracted Three.js into React components we already knew how to deal with, and Drei abstracted common 3D patterns into single-line imports. The 3D rendering was by far the most technically demanding aspect of the frontend, and without this layer of abstractions it would not have been possible within the project's timeframe to implement the core features we wanted for the frontend.

The Components

Having outlined the rendering stack, the following section walks through the main components that were built on top of it. Each component demonstrates how the three layers described above were combined in actual practice.

SceneCanvas The `SceneCanvas` composes the scene:

```
<Canvas camera={{ position: [0, 5, 10], fov: 50 }}>
  <Suspense fallback={<Loader />}>
    <Bounds fit clip observe>
      <Model url={modelUrl} onLoad={handleLoad} />
    </Bounds>
  </Suspense>
  <OrbitControls makeDefault />
  <Environment preset="apartment" />
  <GizmoHelper alignment="bottom-right">
    <GizmoViewport />
  </GizmoHelper>
  <AdaptiveDpr pixelated />
  {bounds && showGrid && !awaitingResults && <VoxelGrid />}
  {bounds && isStaging && <SourcePlacer />}
</Canvas>
```

The `SceneCanvas` component is responsible for the rendering of the 3D Scene. A `<Canvas>` component initialises the WebGL context and camera. Inside it, a `<Suspense>` boundary that wraps the `<Model>` component, showing a loading indicator that keeps track of loaded and pending data. The `Model` component uses **Drei's** `useGLTF` to load the `.glb` file and computes a `THREE.Box3` bounding box. This bounding box matches the dimensions of the AABB computed by `AT_scene_create()`, ensuring that the voxel indices in the response map to correct world positions, producing a valid heat map. Finally `<Bounds>` automatically frames the camera around the loaded geometry, allowing us to not have to worry about scale and orientation of uploaded models. The additional **Drei** components, (`OrbitControls`, `Environment`, `GizmoHelper`, `AdaptiveDpr`) provide interaction, lighting, an orientation helper, and automatic resolution scaling for performance. The two main components are conditionally rendered: `<VoxelGrid />` renders only when the bounding box is calculated, the grid toggle is enabled, and the simulation is not awaiting results. Similarly `<SourcePlacer />` renders only the bounds are known, with its controlled by the `isStaging` prop. An example figure is given below showing the final result of these components working in tandem.

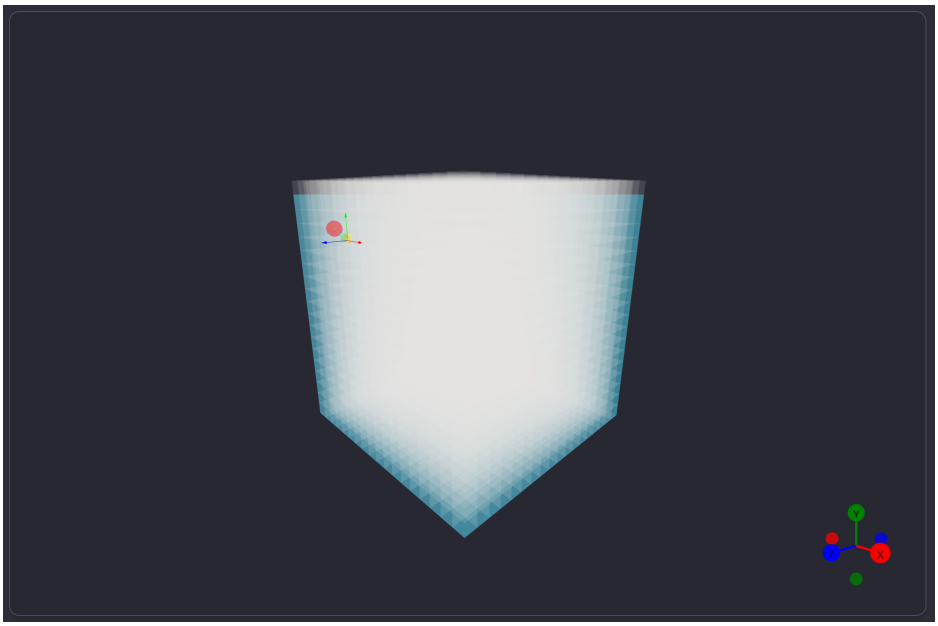

Figure 4: Scene Example

VoxelGrid

The `VoxelGrid` component uses an `InstanceMesh` to render the full 3D voxel grid, an `InstanceMesh` is a special **Three.js** class that draws many copies of the same geometry in a single GPU draw call. The `InstanceMesh` has two rendering modes based on if there is a simulation result loaded or not, if there is no ray response every cell in the grid is rendered as a white, low opacity cube. If there is an ray response then only the voxels present in the `currentFrame.indices` are positioned. We then set the colour of the voxel based off the `energy x numRays`, each voxel's energy is an extremely small float, we multiply these values by numRays to rescale is back to a normalised range between zero and one. We then linearly maps the normalised energy values from that range to the specified hue range.

- Hue 0.66 = blue (low energy)
- Hue 1.0 = red (high energy)

So a voxel with zero energy maps to blue, and a voxel at maximum energy maps to red, as shown below in figure 4.

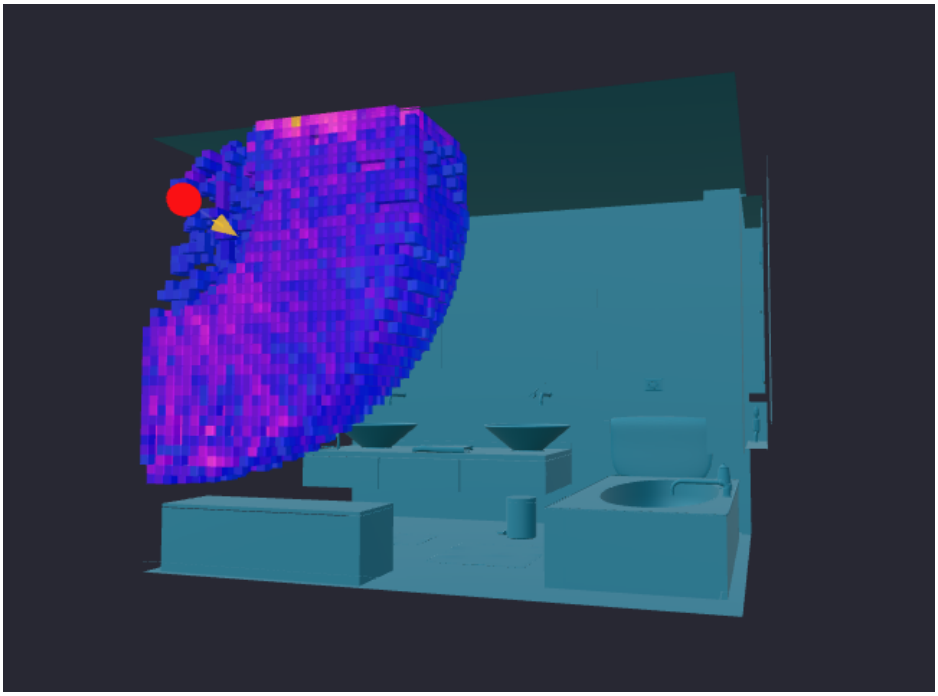

Figure 5: Voxels Heatmap

Grid Dimensions The grid dimensions `nx`, `ny` and `nz` are derived from the scene's bounding box and the user defined voxel size, The dimensions are calculated identically to the C backend, `ciel((max - min) / voxelsize )`. The `gridIndextToPos` callback then converts each voxels flat index back to the actual world space coordinates, which is then offset by the bounding box minimum plus half the voxel size to centre each cube within its cell.

InstanceMesh Optimisation When we create an `InstancedMesh`,Three.js allocates a GPU buffer large enough for exactly the number of voxels. This buffer size is fixed at creation time. For example if frame 1 has 500 active voxels and frame 2 has 12,000, you can't just dynamically grow the buffer instead we have to destroy the mesh and create a new mesh for each frame, which led to visual flickering of the grid and a massive drop in performance. The solution we came up with was to scan every frame in the simulation result and find the frame with the largest amount of active voxels. The mesh is then allocated

that buffer size. This solution led to another optimisation that had to be made, if the buffer size was set to accommodate for 12,000 voxels then at frame 1 when there is only 500 active voxels the question arises how do we hide the 11500 unused voxels? For active voxels we render and position them as normal but for unused voxels we transform their cube geometry to a single point (`mesh.setMatrixAt(i, ZERO_MATRIX)`). The GPU still "draws" these instances, but they produce no pixels, reducing the cost of rendering them to essentially zero.

Source and Direction Markers

The simulation requires two spatial inputs, where the source is and which direction it faces. The `SourcePlacer` component handles both using two **Drei** `TransformControls` components to interactively position two markers. A `useFrame` callback runs on every animation frame, clamping the sources positions to the models bounds. This prevents the markers from being dragged outside the room geometry. To improve performance the position of the two markers are not written to the Zustand store on every frame as this would cause React to re-render 60 times per second. Instead, the final position is commited to the store on mouse release via the tracking of the `dragging-changed` event. Similarly on mouse release the direction vector is normalised to a unit vector, the direction marker snaps back to a fixed distance from the source and the normalised direction is commited to the store.

State Mangement

In the early stages of the project all state lived in `useState` and `useEffect` hooks. This worked while the application was small. But as the component tree grew and more components needed access to the same data, we ran into the classic problem of prop drilling: passing state down multiple levels of nested components just so a deeply nested child could read or update it. To prevent prop drilling we switched to using **Zustand**, a lightweight state management library, that provides a store that can be accessed from any component. However we made the mistake of also putting the server data into the Zustand `scene-store`, this meant for every mutation we made that involved the backend such as creating, deleting or updating a simulation meant we had to manually trigger a refetch and write the fresh data back to the store, which led to UI displaying stale information and numerous bugs. The solution was to use **TanStack Query** to manage all server state and keep **Zustand** for client state.

Zustand

Zustand retained ownership of everything that is purely local to the browser session such as voxel size, UI toggles, the current playback frame index, and the pending upload file. These values change frequently so a cache-based solution would be unnecessary.

TanStack

TanStack Query manages everything that lives in the Appwrite database or is fetched over the network: the simulation list, individual simulation records, and the parsed ray response binaries. Unlike Zustand's synchronous state, this data can become stale at any moment, another tab could delete a simulation, or the C backend could finish a run while the user is on a different page, TanStack Query is designed to handle exactly this functionability.

Authentication and Persistent Storage

Authentication and persistent storage are both provided by Appwrite, a Backend-as-a-Service that exposes three clients initialised in `lib/appwrite.ts`.

-`Account` for authentication. -`TablesDB` for the simulation database. -`Storage` for file uploads.

The `UserProvider` component manages authentication state. It checks for an existing session via `account.get()`. All route components are protected by a `ProtectedRoute` layout that checks the current user and redirects to the login page if unauthenticated. Email/password login, registration, and Google OAuth are exposed through this context. On both login and logout, the provider resets the Zustand SceneStore and clears the TanStack Query cache via `queryClient.clear()` which is then followed by `useSceneStore.getState().reset()` to prevent data from one session leaking to another. Our application of persistent storage serves two purposes, first the user's uploaded `.glb` file is stored in Appwrite storage ready to be reloaded when revisting a saved simulation. Second, the `ATRB` binary result returned by the C backend is uploaded Tanstack Query was then used to fetch and cache the file so on loading a saved simulation the playback would load near instantously.

What we would do different

If we were to begin this project again, there are a number of design decisions we would revisit.

Beam Tracing. Our ray tracing implementation is stochastic, meaning that early specular reflections may be missed or under-sampled at lower ray counts. Beam tracing addresses this by expanding each ray into a volumetric beam covering a solid angle of space, guaranteeing that all specular reflection paths within that angle are found exactly rather than approximated. A hybrid approach, beam tracing for the first few orders of specular reflection, then stochastic ray tracing for the diffuse late reverberation, is the architecture used by tools like ODEON [13] and is identified by Savioja and Svensson as the dominant pattern in modern room acoustic simulation [14]. Incorporating this would have produced more reliable results at lower ray counts, directly reducing computation time.

Conclusion and Future Plans

Some features we wished to add include:

Multiple Source Selection on the Front-End Web Application We have included support for multiple sources in our core library, with the user able to specify an arbitrary number of sources and pass them as an array into the scene configuration. However, due to the time constraint we were unable to implement this functionality in the frontend application. This would have been quite the spectacle in the final demonstration.

Genetic Algorithm for Optimal Source Position[15] One of our most desired features during the design phase of this project was to include a Machine Learning artefact into our solution. We speculated on creating a genetic algorithm to locate the optimal position for a source, and it's direction given the 3D space. This was in the hopes that if the user was constructing an environment, and wanted to find out the best position and direction for an arbitrary source, that our software would provide this functionality. This could also introduce the concept of reinforcement learning in artificial intelligence [16], which essentially rewards a machine learning model upon making optimal choices in a dynamic environment. We all found this topic extremely interesting upon its discovery. However again, due to the time constraint and inherent complexity of this feature, we had to prioritise polishing the already completed features.

LiDar Scanning for 3D Environment [17] Similar to the genetic algorithm for Machine Learning, we wished to include a hardware inspired element for our project. We thought of using a LiDar scanner (a detection system which works on the principle of radar, but uses light from a laser), sometimes found in modern smartphone devices, to allow a user to scan their environment, then upload to our software.

[13] ODEON Room Acoustics Software
[14] Overview of geometrical room acoustic modeling technique
[15] Genetic Algorithm
[16] Reinforcement Learning
[17] LiDar

However this approach came with many caveats for its implementation. Firstly, high quality LiDar scanners are extremely expensive, and were not readily available to us during this project. The LiDar scanner in select mobile phones (most recent Pro and Pro Max models of the iPhone), is not exposed heavily to the end user, and is used internally. Additionally its quality is inferior to the more expensive devices, which causes the environment scanned during its use to be much more 'noisy' and would require a high level of post-processing or smoothing to turn into a usable model. Methods to expose the LiDar functionality do exist, but require a payment subscription or limited free-tier usage. We decided that this method was infeasible to us, along with the end user, and thus decided to go with the 3D modelling approach, which is much more user friendly and accessible.

We were ambitious throughout the duration for this project and had many ideas for features we wished to implement along the way. However, due to the time constraint of the project we were not able to finish every feature we had set our eyes on. We will continue to work on this project after the deadline as it is something we are each proud of and passionate about. We believe we have demonstrated a unique view on an intriguing area of research.